

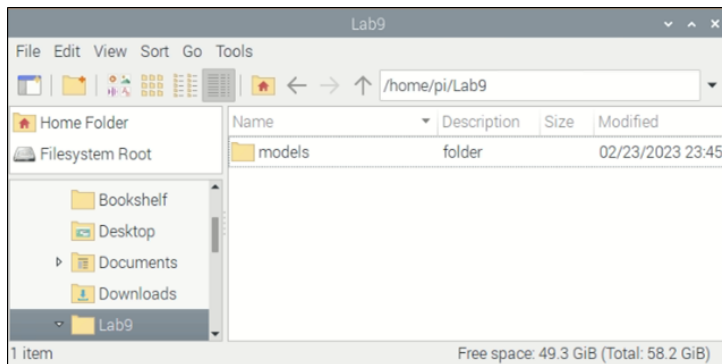
Lab 9.2 Machine Learning 2 - Machine Learning Implementation to Edge Device

TensorFlow implementation to Raspberry Pi

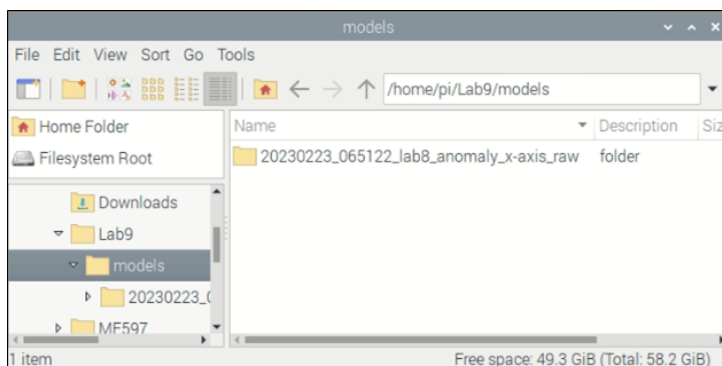
2.1 Load and Run TensorFlow Model

To implement the TensorFlow model which you developed and finally was selected in Lab 8 to Raspberry Pi, please follow the steps below. Please note that all required functions and the basic structure of the script are prepared in [lab9_ML_implementation.py](#).

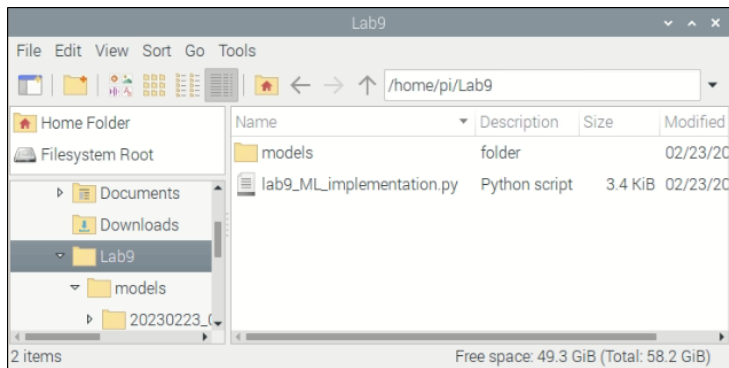
1. Create a working directory on Raspberry Pi.
 - a. In this example, the directory is '/home/pi/Lab9'. Your directory may be different.



2. Copy and paste the trained TensorFlow model directory under '/models' directory of your working directory from laptop.
 - a. In this example, the model directory is '/models/20230223_064122_lab8_anomaly_x-axis_raw'. Your directory must be different.



3. Copy and paste 'lab9_ML_implementation.py' to the same directory of Raspberry Pi as below.



Perform Task 2.1 to get the prediction result from the model and Task 2.2 to complete the algorithm (Figure 2).

Task 2.1

1. Open [lab9_ML_implementation.py](#) on the Raspberry Pi and then modify it according to your ML model requirements
 - a. You need to modify variables surrounded by asteriks in the part of the code below. E.g., `*example_var* = /sample/dir/` -> `example_var = /real/dir/`.
2. Run the modified code, capture 'Terminal' or 'Thonny Shell' as Figure 5 and then attach it to the report.
 - a. By turning the AFF on and off repeatedly, confirm if your model works well.

```
❖ (base) pi@robertclaud:~/Documents/lab $ /home/pi/Documents/lab/lab-venv/bin/python /home/pi/Documents/lab/lab9/lab9_ML_implementation.py
Output data rate is 3200 Hz
Input shape: (1, 1000)
2026-04-05 16:25:36.378858: Model result=True, MAE loss=0.0770
Input shape: (1, 1000)
2026-04-05 16:25:38.364261: Model result=True, MAE loss=0.0782
Input shape: (1, 1000)
2026-04-05 16:25:40.322649: Model result=True, MAE loss=0.0790
Input shape: (1, 1000)
2026-04-05 16:25:42.301485: Model result=True, MAE loss=0.0822
Input shape: (1, 1000)
2026-04-05 16:25:44.259159: Model result=True, MAE loss=0.0837
Input shape: (1, 1000)
2026-04-05 16:25:46.216061: Model result=True, MAE loss=0.0889
Input shape: (1, 1000)
2026-04-05 16:25:48.279273: Model result=False, MAE loss=0.1293
Input shape: (1, 1000)
2026-04-05 16:25:50.236968: Model result=False, MAE loss=0.2033
Input shape: (1, 1000)
2026-04-05 16:25:52.199173: Model result=False, MAE loss=0.2305
```

Python - Python3 ("Main" of [lab9_ML_implementation.py](#))

```
if __name__ == "__main__":
    *model_path* = "models/YourModelDirectory/" # model file directory, you must change
    this!
    model = tf.keras.models.load_model(model_path) # load the model from the path above
    *threshold* = # threshold (MAE loss) for the ML model
    *min_val* = # minimum value for normalization
    *max_val* = # maximum value for normalization

    while True:
        try:
            x, y, z = measureData(acc, 1000) # x=x-axis, y=y-axis, z-axis acceleration
            array
            *input_feature* = # your input feature
            input_feature_normalized = tensorNormalization(input_feature, min_val,
            max_val) # normalized input feature
            result = predict(model, *FINAL_FEATURE_INPUT*, threshold) # your model result:
            bool: True or False
            now = datetime.datetime.now() # datetime now
```

```

        print("{0}:Model result={1}, MAE loss={2:.4f}.".format(now, result[0],
result[1]))
    except KeyboardInterrupt:
        # to halt, press Ctrl + c
        raise

```

Figure 5 The result of the ML model by Task 2.1: 'Terminal' (left) and 'Thonny' (right)

Task 2.2 Answer to the questions below regarding Task 2.1:

1. Does the model work as expected?

Yes, the model works as expected after fine tuning the threshold. It can give a "true" reading when the AFF is in normal operating conditions and false when the AFF is not.

2. What does the result of the model mean?

The result of the model means that there is a certain threshold with which the raspberry pi can interpret sensor readings as normal operating conditions or abnormal operating.

3. When the AFF stop, what is the result of the model?

When the AFF is stopped, the result of the model is "true" similar to when the AFF is operating in normal conditions.

Task 2.3

1. To complete algorithm as Figure 2, modify the code ('lab9_ML_implementation.py') after performing Task 1.1, Task 1.3, and Task 2.1.

a. When the AFF is running, the execution must be 'ACTIVE'.

b. When the AFF is running without the attached putty, the condition must be 'NORMAL'.

c. When the AFF is running with attached putty, the condition must be 'ABNORMAL'.

* **An alternative to make abnormal condition is tapping the AFF. It does make sense that tapping is another case of anomaly for the AFF.**

d. When the AFF is NOT running, the execution and the condition must be 'STOPPED' and 'UNAVAILABLE', respectively.

e. Please note that the algorithm can be expressed as below:

```

if AFF is running:
    execution = 'ACTIVE'
    if AFF is normal:
        condition = 'NORMAL'
    else:
        condition = 'ABNORMAL'
else:
    execution = 'STOPPED'
    condition = 'UNAVAILABLE'

```

2. The example result is shown in Figure 6.

a. If you are not sure about how to determine the execution, review [Task 1.1](#).

```

pi@TA: ~/Lab10
File Edit Tabs Help
X-axis rms=2.4040 m/s2, Y-axis rms=3.7731 m/s2, Z-axis rms=9.8731 m/s2
2022-03-31 04:03:20.455176:Execution=ACTIVE, condition=ABNORMAL, loss=0.0292.
X-axis rms=2.1188 m/s2, Y-axis rms=3.3388 m/s2, Z-axis rms=9.9329 m/s2
2022-03-31 04:03:22.400145:Execution=ACTIVE, condition=ABNORMAL, loss=0.0233.
X-axis rms=2.2386 m/s2, Y-axis rms=2.9837 m/s2, Z-axis rms=9.8086 m/s2
2022-03-31 04:03:23.662899:Execution=ACTIVE, condition=ABNORMAL, loss=0.0243.
X-axis rms=1.9230 m/s2, Y-axis rms=2.7488 m/s2, Z-axis rms=9.8351 m/s2
2022-03-31 04:03:24.938276:Execution=ACTIVE, condition=ABNORMAL, loss=0.0181.
X-axis rms=1.4103 m/s2, Y-axis rms=2.2227 m/s2, Z-axis rms=9.7841 m/s2
2022-03-31 04:03:26.250574:Execution=ACTIVE, condition=NORMAL, loss=0.0083.
X-axis rms=1.3738 m/s2, Y-axis rms=2.6907 m/s2, Z-axis rms=9.7227 m/s2
2022-03-31 04:03:27.567561:Execution=ACTIVE, condition=NORMAL, loss=0.0081.
X-axis rms=1.3221 m/s2, Y-axis rms=2.8479 m/s2, Z-axis rms=9.8053 m/s2
2022-03-31 04:03:28.832148:Execution=ACTIVE, condition=NORMAL, loss=0.0087.
X-axis rms=0.6317 m/s2, Y-axis rms=0.6048 m/s2, Z-axis rms=9.6324 m/s2
2022-03-31 04:03:30.715925:Execution=STOPPED, condition=UNAVAILABLE, loss=0.0540.
X-axis rms=0.6425 m/s2, Y-axis rms=0.6810 m/s2, Z-axis rms=9.5922 m/s2
2022-03-31 04:03:32.686707:Execution=STOPPED, condition=UNAVAILABLE, loss=0.0540.

Shell
2022-03-31 04:00:33.407586:Execution=ACTIVE, condition=NORMAL, loss=0.0086.
X-axis rms=1.3189 m/s2, Y-axis rms=2.2224 m/s2, Z-axis rms=9.6328 m/s2
2022-03-31 04:00:34.650574:Execution=ACTIVE, condition=NORMAL, loss=0.0079.
X-axis rms=1.7820 m/s2, Y-axis rms=2.6547 m/s2, Z-axis rms=9.7815 m/s2
2022-03-31 04:00:35.910878:Execution=ACTIVE, condition=ABNORMAL, loss=0.0159.
X-axis rms=2.1613 m/s2, Y-axis rms=2.7418 m/s2, Z-axis rms=9.8883 m/s2
2022-03-31 04:00:37.170245:Execution=ACTIVE, condition=ABNORMAL, loss=0.0237.
X-axis rms=2.5596 m/s2, Y-axis rms=3.1554 m/s2, Z-axis rms=9.7733 m/s2
2022-03-31 04:00:38.448386:Execution=ACTIVE, condition=ABNORMAL, loss=0.0304.
X-axis rms=1.3639 m/s2, Y-axis rms=2.4270 m/s2, Z-axis rms=9.6708 m/s2
2022-03-31 04:00:40.398979:Execution=ACTIVE, condition=NORMAL, loss=0.0087.
X-axis rms=0.9020 m/s2, Y-axis rms=0.8595 m/s2, Z-axis rms=9.6129 m/s2
2022-03-31 04:00:41.716572:Execution=STOPPED, condition=UNAVAILABLE, loss=0.0269.
X-axis rms=0.8216 m/s2, Y-axis rms=0.8293 m/s2, Z-axis rms=9.6166 m/s2
2022-03-31 04:00:43.015318:Execution=STOPPED, condition=UNAVAILABLE, loss=0.0345.
Python 3.7.3

```

Figure 6 The result of algorithm by TASK4: 'Terminal' (left) and 'Thonny' (right)

```

(base) pi@robertcloud:~/Documents/lab $ /home/pi/Documents/lab/lab-venv/bin/python /home/pi/Documents/lab/lab9/lab9_ML_implementation.py
Output data rate is 3200 Hz
X-axis rms=0.3725 m/s2, Y-axis rms=0.4354 m/s2, Z-axis rms=9.5806 m/s2
Time: 2026-04-05 16:56:58.961786: Execution = STOPPED, Condition = UNAVAILABLE, loss = 0.0772
X-axis rms=0.3509 m/s2, Y-axis rms=0.4422 m/s2, Z-axis rms=9.6114 m/s2
Time: 2026-04-05 16:57:00.947486: Execution = STOPPED, Condition = UNAVAILABLE, loss = 0.0770
X-axis rms=0.4716 m/s2, Y-axis rms=0.5613 m/s2, Z-axis rms=9.5882 m/s2
Time: 2026-04-05 16:57:02.916639: Execution = ACTIVE, Condition = NORMAL, loss = 0.0768
X-axis rms=0.6288 m/s2, Y-axis rms=0.6642 m/s2, Z-axis rms=9.6161 m/s2
Time: 2026-04-05 16:57:04.886017: Execution = ACTIVE, Condition = NORMAL, loss = 0.0804
X-axis rms=1.3194 m/s2, Y-axis rms=0.9057 m/s2, Z-axis rms=9.7153 m/s2
Time: 2026-04-05 16:57:06.865740: Execution = ACTIVE, Condition = NORMAL, loss = 0.0982
X-axis rms=0.9902 m/s2, Y-axis rms=1.6701 m/s2, Z-axis rms=9.6237 m/s2
Time: 2026-04-05 16:57:08.850197: Execution = ACTIVE, Condition = ABNORMAL, loss = 0.1610
X-axis rms=1.0933 m/s2, Y-axis rms=1.6964 m/s2, Z-axis rms=9.6224 m/s2
Time: 2026-04-05 16:57:10.860185: Execution = ACTIVE, Condition = ABNORMAL, loss = 0.1662
X-axis rms=0.9425 m/s2, Y-axis rms=1.4213 m/s2, Z-axis rms=9.5832 m/s2
Time: 2026-04-05 16:57:12.930064: Execution = ACTIVE, Condition = ABNORMAL, loss = 0.1353
X-axis rms=0.3747 m/s2, Y-axis rms=0.4438 m/s2, Z-axis rms=9.6018 m/s2
Time: 2026-04-05 16:57:14.902547: Execution = STOPPED, Condition = UNAVAILABLE, loss = 0.0778
X-axis rms=0.3702 m/s2, Y-axis rms=0.4446 m/s2, Z-axis rms=9.5852 m/s2
Time: 2026-04-05 16:57:16.878883: Execution = STOPPED, Condition = UNAVAILABLE, loss = 0.0770

```

Task 2.4 Do your model and algorithm work as expected? Try change the running states (executions) and conditions.

- If yes, please explain your test setup and the results.
- If not, please explain the possible reasons and how to make it work.

Yes, the test setup worked. After fine-tuning my MAE threshold and implementing the rms threshold I set up the model and algorithm to work such that:

- If the rms threshold was exceeded the execution was set to "ACTIVE". Otherwise, the execution was set to "STOPPED".
- If the model result was "true" and the execution was "ACTIVE", the condition was set to "NORMAL"

- If the model result was "false" and the execution was "ACTIVE", the condition was set to "ABNORMAL"
- If the execution was "STOPPED" then the condition was set to "UNAVAILABLE".

The setup was tested under three conditions.

1. The fan was turned off
 - This returned a "STOPPED" execution and "UNAVAILABLE" condition.
 2. The fan was turned on under normal operating conditions
 - This returned an "ACTIVE" execution and "NORMAL" condition.
 3. I placed my finger on the fan to avoid having to scrape putty off the fan over and over.
 - This returned an "ACTIVE" execution and "ABNORMAL" condition.
-

Please continue to [Lab 9.3 here](#).